B. VIJAY RAMAN

Engineer + Product | Customer Discovery to Shipped AI | Agentic and Real-Time Voice AI

I sit with the customer, write the spec, then build and deploy it. 4+ years of product ownership at an enterprise contact-center platform, the last 2+ owning AI products end to end across 5 enterprise clients. M.Sc. Computer Science, NIT Trichy.

Noida, India | Remote | ramanvijay2878@gmail.com | linkedin.com/in/vijay-raman-a6b947178 | github.com/Vijayraman1998

PROFESSIONAL SUMMARY

Engineer and product owner: 4+ years turning ambiguous enterprise problems into shipped software, the last 2+ owning AI products end to end. I run the customer conversation myself: discovery with client product, operations and compliance stakeholders; the specification, tool contracts and success criteria; then the code, the integration into their customer relationship management (CRM) and telephony stack, and the iteration on live traffic after go-live. My voice agents handle around 800,000 call-pipeline events a day at 70%+ automated containment, across 5 enterprise clients in lending, automotive and sales.

Product judgment and engineering are the same job for me. I decide what to build from evaluation data rather than opinion, and I have killed my own proposals when the data disagreed. I work in Java and Spring on Azure OpenAI with self-built retrieval-augmented generation (RAG), guardrails and evaluations, and I ship day to day with agentic coding tools, reviewing and debugging generated code rather than trusting it.

ACHIEVEMENTS

- **Shipped 15+ production agent deployments across 5 enterprise clients:** in lending, automotive and sales (names under NDA): multi-agent systems with autonomous function calling, 7+ typed tools per agent and live customer relationship management (CRM) reads and writes mid-conversation.
- **800,000+ call-pipeline events per day at 70%+ containment:** conversations resolved with no human transfer, at under 500 ms to first audio, measured from final transcript to first audio byte, peaking near 500 concurrent live calls.
- **Deployments cut cost per conversation 50-80% against a human agent and shortened average handle time 10-30% on most use cases:** the business case each rollout was bought on, modeled per client on that client's own call mix.
- **Cut time-to-value from about 2 weeks to hours:** agent delivery moved from a services engagement to a self-serve product surface that client teams configure themselves.
- **Ran each engagement end to end as the single technical owner:** discovery with the client's product, operations and compliance stakeholders, then the specification, the build, the integration into their CRM and telephony, UAT, go-live and post-launch tuning on live traffic.
- **Reversed my own proposal on evaluation data:** team consensus, mine included, was that we needed a larger model; the evaluation framework I built showed that the failures were in retrieval. Fixing retrieval lifted answer faithfulness 35% against the prior baseline at flat inference cost, and avoided an upgrade that would have cost more and fixed less.

PROFESSIONAL EXPERIENCE

Product Owner and Engineer, AI Platform

Rezo.AI, Noida, India | Jan 2024 - Present

Own the AI agent product end to end: customer discovery, specification, build, deployment and iteration

Customer discovery and product definition

- **Led discovery directly with the client's product, operations and compliance stakeholders** who arrived with a business goal and no written specification, and converted that goal into agent scope, tool contracts, escalation rules and measurable success criteria before a line of code was written.
- **Set the success metric before the build, not after**: containment rate, average handle time and cost per conversation, instrumented from day one, so the pilot produced a defensible go/no-go decision rather than an impression. I paired containment with explicit escalation triggers, because an agent that never hands off scores well and serves the caller badly; containment counts only when the caller did not need a human.
- **Negotiated scope against delivery reality with founders and client executives**, cutting the launch to the two intents that carried the volume and deferring the long tail, which pulled first go-live forward and gave the remaining intents real traffic data to design against.
- **Remained the technical point of contact after go-live**, reading production transcripts weekly and folding recurring failure modes back into the roadmap, so each deployment started further ahead than the last.

Real-time voice agent product

- **Owned the real-time voice agent product end to end** across scope, runtime and interface: a streaming STT to LLM to TTS pipeline with barge-in, interruption handling and turn-taking, holding time-to-first-audio under 500 ms, measured from final transcript to first audio byte, on a platform carrying around 800,000 call-pipeline events per day and peaking near 500 concurrent live calls.
- **Solved the latency problem with engineering rather than spend**: instrumenting each stage separately showed that LLM first-token latency dominated, so I shipped end-to-end streaming where synthesis starts on the first completed sentence rather than upgrade to a faster, costlier model. Same model, same cost.
- **Shipped pluggable multi-provider TTS (OpenAI, Azure, ElevenLabs, Google)** selectable per agent, with model tiering by stakes, so routine turns use a cheap, fast voice and high-value turns a heavier one, a cost lever the customer controls.

From services engagement to self-serve product

- **Diagnosed the constraint as delivery, not demand**: every new agent needed an engineer, so the roadmap was capped by headcount. I specified and drove a self-serve builder that lets client teams configure agents, prompts and tools themselves.
- **Cut time-to-value from about 2 weeks to hours** and moved engineering effort off repeat delivery and onto the platform, changing the economics of each new client rather than the speed of each new ticket.

- **Designed and deployed 15+ production agents on Azure OpenAI GPT** across 5 enterprise clients, with typed tool schemas per agent and structured JSON output validated before any CRM write.

Evidence over opinion: evaluation and a reversed decision

- **Wrote the production evaluation framework and quality dashboards** on labeled sets drawn from real client traffic, scoring retrieval and generation separately, so neither could hide behind a single accuracy number.
- **The evaluation overturned the internal consensus, including mine**: we believed we needed a larger model; the data showed that most failures were in retrieval. Fixing retrieval lifted answer faithfulness 35% across 5 clients at flat inference cost and avoided an upgrade that would have cost more and fixed less.
- **Built the retrieval-augmented generation (RAG) layer on LLM embeddings** on a Java stack, and tuned it per client against that client's own labeled set.

Productizing the build itself

- **Designed a closed-loop agent build pipeline**: a requirements document goes in, an agent drafts the system prompt, tool contracts, service code and schema, synthesizes its own test cases including edge cases, runs LLM-driven conversation simulations and scores them, and routes failures back to the prompt-writing stage until the suite is green.
- **Held a deliberate human release gate** at the end of that loop: a QA sanity pass, then client UAT, before production. Automating the work is a speed decision; automating the accountability would not have been.
- **Extended the same agentic tooling to QA and to production voice-call quality control**, scoring live calls for compliance, accuracy and escalation quality instead of sampling by hand.
- **Ship day to day with agentic coding tools** (Claude Code, Codex), treating generated code as a pull request from a fast junior engineer: reviewed, tested and debugged by handing the agent a failing test rather than shipped on trust.

Scale and reliability

- **Engineered thread-safe multi-turn session management** with automatic cleanup, token tracking and context-window limits, holding 500+ concurrent conversations with per-session isolation.
- **Scaled a multi-tenant WhatsApp queue on RabbitMQ** across 200+ tenant accounts with P1/P2/P3 routing, 50-thread concurrency and tenant isolation, with zero throttling over 60+ days and 3x peak traffic absorbed at full SLA.

Product Owner and Engineer, Platform and Integrations

Rezo.AI, Noida, India | Aug 2022 - Dec 2023

Ran the integration and platform layer that the AI products are deployed on

- **Owned the integration surface that every customer deployment lands on**: 13+ external systems (Azure OpenAI, telephony providers, client CRMs, AWS SES, interactive voice response (IVR), SFTP) authenticated over JSON Web Token (JWT) and OAuth2, scoped with the client teams on the other side of each integration.
- **Delivered 85+ REST APIs powering the agent backends** with connection pooling and caching for sub-millisecond lookups under production contact-center load, and owned CI/CD across the Java microservice estate, cutting deployment failures 30%.
- **Cut query time 50%** via FORCE INDEX, date partitioning and JOIN refactoring, and built batch pipelines that lifted ingestion throughput 4x.
- **Specified and led delivery of a full-stack QA Automation Platform** (Spring Boot 3, React 18, TypeScript, Spring Security, Spring Data JPA, Vite) to standardize regression testing across the product suite, writing the requirements and mentoring 3 junior engineers through delivery.

SKILLS

GenAI and Agentic Systems: Azure OpenAI GPT API, Prompt engineering and layered prompt architecture, LLM function and tool calling, Typed tool schemas and structured JSON output, Multi-agent orchestration, Retrieval-augmented generation (RAG) with LLMs, Output guardrails and escalation triggers, Context-window and token-cost management

Evaluation and Quality: Offline evaluation sets built from production traffic, Scoring with an LLM as judge, Quality dashboards and regression gating, Production voice-call quality control

Real-Time Voice AI: Streaming speech-to-text (STT) to LLM to text-to-speech (TTS) pipelines, Barge-in and turn-taking, Latency tuning to first audio, Multi-provider TTS (OpenAI, Azure, ElevenLabs, Google), Conversational AI and contact center as a service (CCaaS)

Product: Customer discovery and problem framing, Product requirements documents (PRDs) and specifications, Roadmap and prioritization, Success metrics and instrumentation, Backlog, epics, user stories and acceptance criteria, Pilot and proof-of-concept (POC) scoping, UAT and release management, Agile and Scrum delivery

Customer Ownership: Client-facing delivery and solutions engineering, Founder and executive stakeholder communication, Production go-live and service-level agreement (SLA) ownership, Regulated BFSI requirements (consent, escalation and audit)

Engineering: Java 8/17, Spring Boot, REST API and distributed-system design, Microservices, Concurrency, RabbitMQ, Kafka, MySQL, PostgreSQL, Docker, AWS, Azure, CI/CD, React 18, TypeScript, Linux

AI-Assisted Development: Claude Code and Codex as the primary build loop, Critical review and debugging of AI-generated code, Spec-driven and test-driven prompting, Model Context Protocol (MCP)

EDUCATION

Master of Science (M.Sc.), Computer Science, National Institute of Technology Tiruchirappalli, India | 2020 - 2022

Bachelor of Science (B.Sc.), Statistics, University of Delhi, India | 2017 - 2020